

Data Diode Based File Transfer Simulator

A Python simulation of a hardware data diode — a device that enforces **one-way data flow** between network zones using cryptographic controls, antivirus scanning, a strict state machine, and an optional Time-Division Duplex (TDD) mode for controlled bidirectional transfer.

Designed for defence, SCADA, and air-gapped network contexts where data must move from an Internet-connected zone into a secure offline zone with zero possibility of reverse exfiltration.

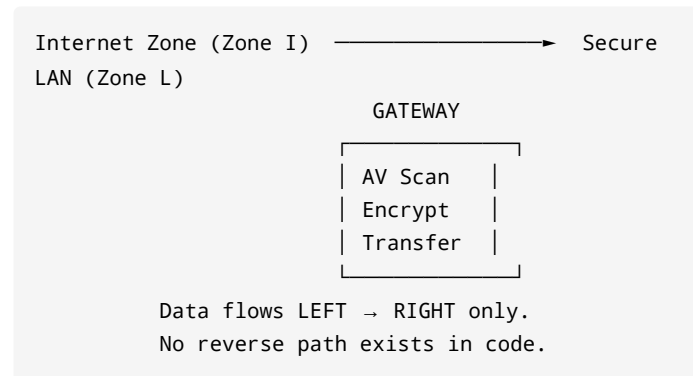
Table of Contents

- [Concept](#)
- [Architecture](#)
- [Security Model](#)
- [State Machine](#)
- [Module Structure](#)
- [Directory Layout](#)
- [Quick Start](#)
- [Usage — GUI](#)
- [Usage — CLI](#)
- [TDD Bidirectional Mode](#)
- [Configuration Reference](#)
- [Cryptography](#)

- [Logging and Audit Trail](#)
 - [Deployment on Raspberry Pi](#)
 - [Limitations](#)
-

Concept

A real hardware data diode is a physical device with a fibre-optic transmitter on one end and no receiver – light can only travel in one direction. This project simulates that behaviour in software:



Defence relevance: Used in military, nuclear, and critical infrastructure networks where even a software path back to the Internet is unacceptable.

Architecture

Three-Zone Security Model

Zone	Network	Role
Zone I	Online	Source — drops files into <code>zones/zone_i/input/</code>
Gateway	Isolated processor	Scans, encrypts, enforces state machine
Zone L	Air-gapped / Offline	Destination — receives <code>.diode</code> files, decrypts on request

Forward Pipeline (Zone I → Zone L)

```

Zone I input/
|
▼ [1] File detected by watchdog
|
▼ [2] 1s settle delay (OS flush)
|
▼ [3] Move to gateway/processing/
|
▼ [4] AV scan (simulated or ClamAV)
      └─ FAIL → quarantine/ + ERROR state
|
▼ [5] AES-256-GCM encrypt →
gateway/encrypted/<file>.diode
|
▼ [6] Copy to zone_l/incoming/ + SHA-256
integrity verify
|
▼ [7] Delete all gateway staging files
|
▼ [8] Dead time (3s both zones locked)
|
▼ [9] IDLE — ready to decrypt

```

Key Principle

Data can only move LEFT → RIGHT. No reverse path exists in the code.

Every state in the gateway locks one or both zones. It is structurally impossible for a file to travel from Zone L back to Zone I through the forward pipeline.

Security Model

What is enforced

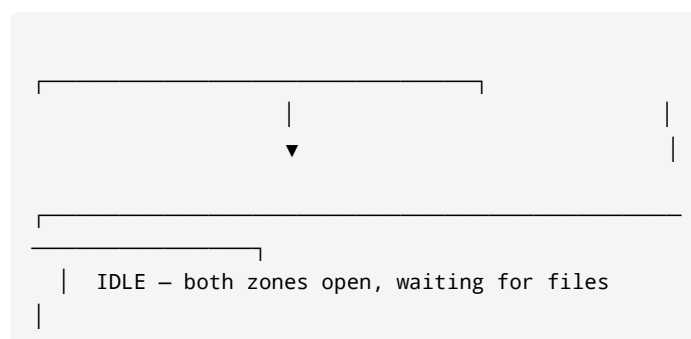
Control	Mechanism
One-way transfer	<code>GatewayState</code> locks zones – <code>_assert_zone_i/l_allowed()</code> guards every cross-zone call
Authenticated encryption	AES-256-GCM – any tampered file raises <code>InvalidTag</code> before plaintext is produced
Password hardening	PBKDF2-HMAC-SHA256 at 390,000 iterations (OWASP 2023 minimum)
Integrity verification	SHA-256 hash of source compared against destination after every copy
Threat quarantine	AV scan failures move files to <code>gateway/quarantine/</code> with timestamped filename
Dead time	3-second hard blackout between pipeline phases – both zones locked

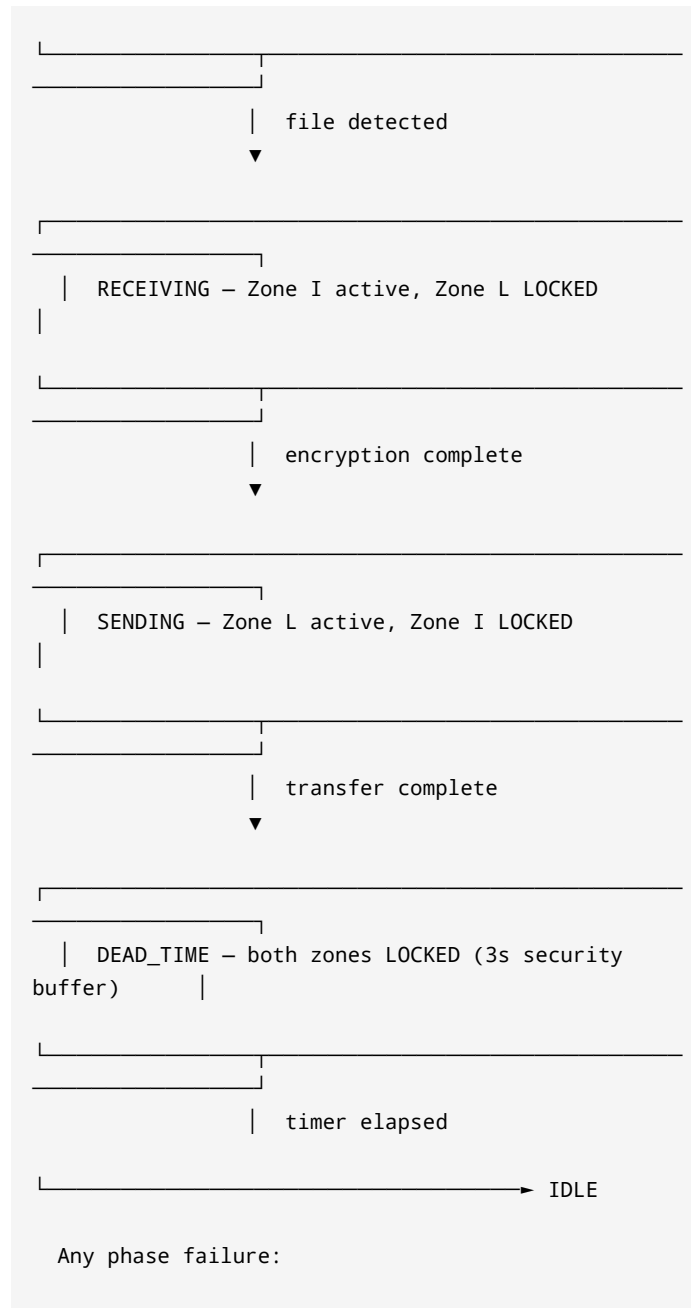
Control	Mechanism
Audit trail	Write-once append-only JSON-lines log at <code>logs/audit.log</code>
Orphan cleanup	Startup sweep quarantines files left by previous crashes
File size limit	512 MB hard cap – configurable in <code>config.py</code>
Password memory	Password string zeroed after PBKDF2 key derivation

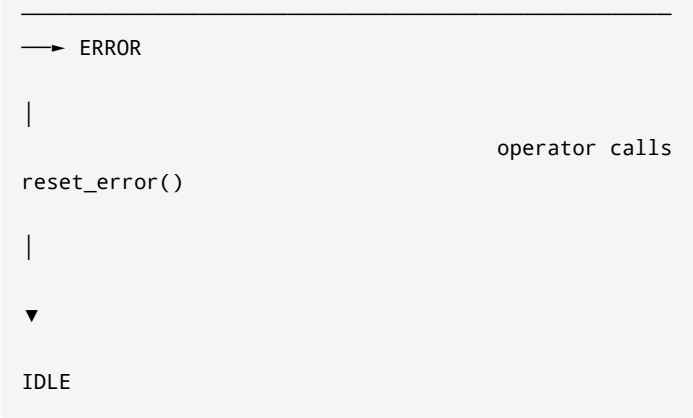
What is NOT enforced (software simulation limits)

- No physical unidirectional link – a compromised OS process could theoretically bypass the state machine
- No network isolation – this simulates the diode via filesystem paths, not actual network interfaces
- Covert channel attacks (timing, power) are not defended against at the software level

State Machine







TDD mode adds four additional states: `SLOT_B_RECV`, `SLOT_B_SEND`, `DEAD_TIME_2`, and a second `DEAD_TIME` between direction switches. Only one state is active at a time – enforced by `threading.Lock`.

Module Structure

Existing modules (v2 – hardened)

File	Responsibility	Key changes in v2
<code>config.py</code>	All constants, paths, <code>GatewayState</code> enum	Added TDD constants, reverse paths, audit log path, <code>MAX_FILE_SIZE_BYTES</code>
<code>gateway.py</code>	State machine + pipeline orchestrator	Dead time enforced; <code>reset_error()</code> quarantines in-flight files; reverse pipeline added

File	Responsibility	Key changes in v2
<code>crypto_utils.py</code>	AES-256-GCM encrypt / decrypt + PBKDF2	File size guard; password zeroed after KDF; returns SHA-256 hash
<code>scanner.py</code>	AV scan — simulated or real ClamAV	<code>is_scanner_error</code> flag; MIME-type check; Shannon entropy check; audit on every scan
<code>transfer.py</code>	Zone delivery + gateway cleanup	SHA-256 integrity check after every copy; <code>deliver_to_zone_i()</code> for reverse; explicit cleanup logging
<code>watcher.py</code>	Monitors zone input folders	<code>on_modified</code> handler with mtime dedup; <code>ZoneLWatcher</code> for TDD
<code>logger_setup.py</code>	Rotating file + console + UI log handler	Thread-safe UI callback; <code>LOG_LEVEL</code> from config
<code>app.py</code>	Tkinter 3-panel GUI + threading model	Password captured on main thread before upload then cleared; queue integration; TDD panel

File	Responsibility	Key changes in v2
<code>cli_runner.py</code>	Headless CLI for servers / CI	<code>argparse</code> ; <code>SIGTERM</code> handler; queue integration; <code>--tdd</code> , <code>--headless</code> , <code>--password</code> flags
<code>setup_folders.py</code>	One-time zone directory initialiser	Creates all v2 directories including reverse pipeline paths

New modules (v2)

File	Responsibility
<code>audit_log.py</code>	Write-once append-only JSON-lines security audit trail
<code>integrity.py</code>	SHA-256 <code>DeliveryReceipt</code> — hashes before copy, verifies after, raises <code>IntegrityError</code> on mismatch
<code>file_queue.py</code>	Thread-safe <code>FileTransferQueue</code> decoupling watchdog from gateway; single consumer thread; back-pressure warning
<code>startup_check.py</code>	Scans staging dirs on boot; quarantines orphaned files from previous crashes
<code>scheduler.py</code>	TDD slot manager — FIXED, PRIORITY, and DEMAND scheduling strategies

Directory Layout

```
project/
├─ zones/
│   └─ zone_i/
│       └─ input/           ← Drop files here
(Zone I → Zone L)
│       └─ incoming/       ← Encrypted files
arriving from Zone L (TDD)
│       └─ decrypted/      ← Decrypted output
from Zone L (TDD)
│   └─ gateway/
│       └─ processing/     ← Forward
scan/encrypt staging (temp – auto-cleared)
│       └─ encrypted/      ← Forward encrypted
staging (temp – auto-cleared)
│       └─ rev_processing/ ← Reverse
scan/encrypt staging (TDD)
│       └─ rev_encrypted/  ← Reverse encrypted
staging (TDD)
│       └─ quarantine/     ← Flagged, infected,
or orphaned files
│   └─ zone_l/
│       └─ incoming/       ← Encrypted .diode
files awaiting Zone L decryption
│       └─ decrypted/      ← Final plaintext
output from Zone I
│       └─ output/         ← Drop files here
(Zone L → Zone I, TDD only)
├─ logs/
│   └─ diode.log           ← Rotating
operational log (5 MB × 3 backups)
│   └─ audit.log           ← Write-once
security audit trail (JSON lines)
├─ app.py                 ← GUI entry point
├─ cli_runner.py           ← CLI entry point
└─ setup_folders.py        ← First-run
```

```
directory setup
└─ requirements.txt
```

Quick Start

1. Install dependencies

```
pip install -r requirements.txt
```

Optional – enables MIME-type content checking in the scanner:

```
pip install python-magic
```

2. Create zone directories

```
python setup_folders.py
```

3. Run the GUI

```
python app.py
```

4. Or run headless (CLI)

```
python cli_runner.py --password mysecretpassword
```

Usage — GUI

The GUI has three panels side by side.

Zone I panel (left)

1. Enter an encryption password in the password field.
2. Click **Browse & Upload** and select any file.
3. The file is copied to `zones/zone_i/input/` and the password is captured.
4. The watchdog detects the new file and the pipeline starts automatically.

Gateway panel (centre)

- Shows live state (IDLE / RECEIVING / SENDING / DEAD_TIME / ERROR).
- Colour-coded zone access badges show which zones are locked.
- Activity log streams all pipeline events in real time.
- **Reset** button clears ERROR state (quarantines any in-flight files first).

Zone L panel (right)

1. When a `.diode` file appears in the list, the transfer is complete.
2. Click **Refresh** to update the list.
3. Select the file, enter the decryption password, and click **Decrypt Selected**.
4. The plaintext file appears in `zones/zone_l/decrypted/`.

Usage — CLI

```
usage: cli_runner.py [-h] [--password PASSWORD] [--tdd] [--strategy {FIXED,PRIORITY,DEMAND}] [--headless]
```

```
options:
  --password, -p    Encryption password (prompted per
file if omitted)
  --tdd             Enable TDD bidirectional mode
(requires TDD_ENABLED=True in config.py)
  --strategy        TDD scheduling strategy: FIXED |
PRIORITY | DEMAND (default: DEMAND)
  --headless        Non-interactive – requires --
password; suitable for CI / systemd
```

Examples:

```
# Interactive, forward only
python cli_runner.py

# Non-interactive forward pipeline
python cli_runner.py --password s3cr3t

# Bidirectional TDD, demand-triggered
python cli_runner.py --tdd --strategy DEMAND --
password s3cr3t

# Fully headless – for systemd service or CI
python cli_runner.py --headless --password s3cr3t
```

CLI commands (interactive mode):

Command	Action
<code>status</code>	Show gateway state and pending files
<code>decrypt</code>	Select and decrypt a file in zone_i/incoming
<code>decrypt-i</code>	Select and decrypt a file in zone_i/incoming (TDD)

Command	Action
<code>reset</code>	Clear ERROR state
<code>quit</code>	Shutdown cleanly

Running as a systemd service:

```
[Unit]
Description=Data Diode Transfer Service
After=network.target

[Service]
ExecStart=/usr/bin/python3 /opt/diode/cli_runner.py
--headless --password YOUR_PASSWORD
WorkingDirectory=/opt/diode
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

The service handles `SIGTERM` cleanly — watchers and queues shut down before exit.

TDD Bidirectional Mode

By default the system is strictly one-way (Zone I → Zone L). TDD mode adds a controlled return path.

Enable TDD

In `config.py`:

```
TDD_ENABLED = True
```

How it works

TDD (Time-Division Duplex) gives each direction its own time slot. Only one direction can be active at any moment — enforced by a global mutex. A hard dead time separates every slot transition.

```
|— Slot A (I→L) —|— Dead time —|— Slot B
(L→I) —|— Dead time —|
    Zone I active      both locked      Zone L active
    both locked
```

Scheduling strategies

Strategy	Behaviour	Use case
DEMAND	A slot only opens when a file is waiting	Most efficient — default
FIXED	Alternates A → B → A → B on a wall-clock interval	Predictable, deterministic
PRIORITY	Zone I gets 3 slots before Zone L gets 1	Asymmetric load (more I → L traffic)

Security note

TDD mode **does not create a true hardware diode**. It creates a *time-scheduled cross-domain gateway* — the return path exists in code. Both directions use the same AES-256-GCM + PBKDF2 pipeline and the same AV scan. The dead-time buffer and mutex prevent simultaneous bidirectional activity, but an attacker who controls the OS could potentially abuse the return slot.

For higher assurance, use two separate unidirectional diode instances rather than TDD mode.

Configuration Reference

All settings are in `config.py` . No environment variables are needed.

Constant	Default	Description
<code>MAX_FILE_SIZE_BYTES</code>	512 MB	Files larger than this are rejected before encryption
<code>FILE_SETTLE_DELAY</code>	1.0 s	Wait after file creation before processing (OS flush)
<code>FILE_QUEUE_MAX_DEPTH</code>	10	Warn when this many files are waiting in the queue
<code>KDF_ITERATIONS</code>	390,000	PBKDF2 iterations — lower to ~100,000 on Raspberry Pi
<code>SLOT_DEAD_TIME_SECONDS</code>	3.0 s	Hard blackout between pipeline phases

Constant	Default	Description
<code>TDD_ENABLED</code>	<code>False</code>	Set <code>True</code> to enable bidirectional TDD mode
<code>SIMULATED_SCAN</code>	<code>True</code>	<code>False</code> to use real ClamAV
<code>LOG_LEVEL</code>	<code>"DEBUG"</code>	<code>"WARNING"</code> for production / Pi deployments
<code>LOG_MAX_BYTES</code>	<code>5 MB</code>	Rotating log size before rollover
<code>LOG_BACKUP_COUNT</code>	<code>3</code>	Number of rotated log files to keep

Cryptography

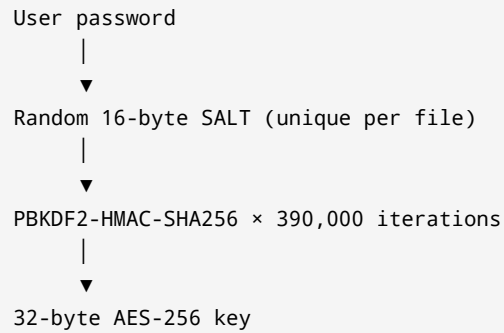
Algorithm: AES-256-GCM

AES-256-GCM provides **confidentiality, integrity, and authenticity** in a single operation.

- Any bit-flip in the ciphertext raises `InvalidTag` before any plaintext is returned.
- Wrong password → invalid GCM tag → `ValueError` raised immediately.

Key derivation: PBKDF2-HMAC-SHA256

Passwords are never used directly as keys.



390,000 iterations makes brute-force cracking ~390,000× slower than a direct dictionary attack.

.diode file format

MAGIC	5 bytes	b"DIODE" – header guard
SALT	16 bytes	random, unique per file
NONCE	12 bytes	random, unique per file
CIPHERTEXT	variable	AES-GCM encrypted payload
GCM TAG	16 bytes	authentication tag

Total overhead: 49 bytes per file.

Same file + same password → DIFFERENT ciphertext every time.

Logging and Audit Trail

Operational log — logs/diode.log

Rotating file handler. Captures all pipeline events, state changes, and errors. Also streamed to console and the GUI log panel.

```
2025-06-12 14:23:01 [INFO ] gateway Gateway
state → RECEIVING
2025-06-12 14:23:02 [INFO ] scanner Scanning:
report.pdf [mode=SIMULATED]
2025-06-12 14:23:02 [INFO ] scanner Simulated
scan PASSED for 'report.pdf'
2025-06-12 14:23:03 [INFO ] crypto Encrypted
'report.pdf' → 'report.pdf.diode'
2025-06-12 14:23:03 [INFO ] gateway Gateway
state → SENDING
2025-06-12 14:23:03 [INFO ] integrity
Integrity OK — report.pdf.diode
2025-06-12 14:23:06 [INFO ] gateway Gateway
state → IDLE
```

Audit log — logs/audit.log

Write-once, append-only, JSON lines. Never rotated or truncated. Records every security-relevant event:

```
{"ts":"2025-06-12T14:23:01.042Z","event":"STATE_CHANGE","old":"IDLE","new":"RECEIVING"}
{"ts":"2025-06-12T14:23:02.187Z","event":"SCAN_RESULT","filename":"report.pdf","passed":true}
{"ts":"2025-06-12T14:23:03.391Z","event":"DELIVERY_OK","filename":"report.pdf.diode","sha256":"a3f9..."}
{"ts":"2025-06-
```

```
12T14:23:03.412Z", "event": "PIPELINE_COMPLETE", "filename": "report.pdf.diode"}
{"ts": "2025-06-
12T14:23:06.415Z", "event": "STATE_CHANGE", "old": "DEAD_TIME", "new": "IDLE"}
```

Events recorded: STATE_CHANGE, SCAN_RESULT, QUARANTINE, DELIVERY_OK, DELIVERY_MISMATCH, PIPELINE_COMPLETE, DECRYPT_ZONE_L, ORPHAN_QUARANTINED, STARTUP_CHECK_ORPHANS, ERROR_RESET, TDD_SLOT_A_START, TDD_SLOT_B_START .

Deployment on Raspberry Pi

The system is designed to run on a Raspberry Pi 4 with two network interfaces — one connected to the Internet, one to the LAN.

Performance tuning

In `config.py`, lower the KDF iteration count to keep encryption latency under 2 seconds:

```
KDF_ITERATIONS = 100_000    # Pi 4 – acceptable
latency
LOG_LEVEL       = "WARNING" # Reduce disk writes
```

Systemd service

```
sudo cp diode.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable diode
sudo systemctl start diode
```

iptables — enforce at OS level

To harden the simulation with OS-level packet filtering
(preventing any actual TCP traffic from LAN back to Internet):

```
# Block all traffic from zone_l network back to
zone_i network
iptables -A FORWARD -s 192.168.2.0/24 -d
192.168.1.0/24 -j DROP

# Allow zone_i → zone_l (gateway processes this at
application layer)
iptables -A FORWARD -s 192.168.1.0/24 -d
192.168.2.0/24 -j ACCEPT
```

Limitations

Limitation	Detail
Software-only	A compromised OS process can bypass the state machine. Use iptables + physical interface separation for defence-in-depth.
Filesystem-based transfer	Files move via <code>shutil.copy2()</code> , not actual network sockets. For real network diode simulation, the transfer layer needs UDP (no TCP — TCP requires bidirectional ACKs).
Simulated AV scan	With <code>SIMULATED_SCAN=True</code> , only filenames are checked against a pattern list. Set <code>SIMULATED_SCAN=False</code> and install ClamAV for real content inspection.

Limitation	Detail
Large file memory	Files are fully loaded into RAM for encryption. The 512 MB cap guards against OOM. Streaming encryption (chunked AES-GCM) is the Phase 3 upgrade path.
TDD is not a true diode	With <code>TDD_ENABLED=True</code> , a return path exists in code. Real cross-domain solutions use two separate unidirectional hardware diodes.
Single password for both directions	In TDD mode, both pipelines use the same password provider. A production system would use separate key management per direction.

Project Roadmap

Phase	Status	Goal
Phase 0	 Complete	One-way pipeline, AES-256-GCM, state machine, GUI + CLI
Phase 1	 Complete	Dead time, SHA-256 receipts, audit log, startup cleanup, queue
Phase 2	 Complete	TDD scheduler, reverse pipeline, 8-state machine

Phase	Status	Goal
Phase 3	☐ Planned	UDP-based transfer, streaming crypto, content-aware scanner, Pi deployment scripts